

OBSERVATIONS

- Overhead Analysis

- 1. Byte Stuffing

Overhead (%) = ((Framed Size - Payload Size) / Payload Size) × 100

Random Data:

- Payload size = 100 bytes
- Framed size = 105 bytes
- Overhead = 5 %

Worst Case:

- Input = 7E 7E 7E ... 7E
- Payload size = 100 bytes
- Framed size = 202 bytes
- Overhead = 102%

Justification: Every payload byte is FLAG, so each requires an additional ESC byte.

- 1. Bit Stuffing

Overhead (%) = ((Stuffed Bits - Original Bits) / Original Bits) × 100

Random Data:

- Payload size = 100 bytes
- Original bits = 800
- Stuffed bits = 809 bits
- Overhead = 1.125 %

Worst Case:

- Input = FF FF FF ... FF
- Original bits = 800
- Stuffed bits = 960
- Overhead = 20%

Justification: FF contains only 1 bits, producing the maximum number of stuffed 0 bits.

- 1. Worst-Case Comparison

- Byte stuffing: 7E 7E 7E ... :- 102% overhead
- Bit stuffing: FF FF FF ... :- 20% overhead

Therefore, byte stuffing has the larger worst-case overhead.

- Byte Count Framing

Original Data payloads : [11 22 33], [AA]

Original Stream : 03 11 22 33 01 AA

The first byte 03 indicates that the first payload contains 3 bytes: [11 22 33]

The second frame starts with 01, indicating a payload of 1 byte: [AA]

During transmission, the first count byte is corrupted from 03 to 05. Therefore, the receiver receives: 05 11 22 33 01 AA

The receiver interprets 05 as the length of the first frame and therefore reads the next 5 bytes: [11 22 33 01 AA]

This consumes the bytes belonging to both frames as one payload. After this, there are no bytes left to identify the boundary of the second frame.

Number of correctly recovered frames: 0 out of 2

In byte-count framing, the receiver depends on the count byte to determine the length of each frame. If a count byte is corrupted, the receiver reads the wrong number of bytes and loses synchronization. Therefore, it cannot correctly determine the boundary of the following frame.